Example of Analysis Failure with Cascading Errors

Phase III – Granite 3.3 – Alert 75 – Analysis Repetition 2

The color-coded markings distinguish between the different error tags and the specific sections that the LLM Judge referenced in its analysis and when assigning the specific error tag.

Alert

"{\n \"timestamp\": \"2025-09-22T14:45:10Z\",\n \"severity\": \"critical\",\n \"source\": \"ModSecurity\",\n \"rule_id\": \"942100\",\n \"message\": \"SQL Injection Attack Detected (Libinjection Check).\",\n \"details\": {\n \"host\": \"WEB-PAYMENTS-01\",\n \"client_ip\": \"45.134.144.12\",\n \"http_method\": \"GET\",\n \"uri_path\": \"/products/details.php\",\n \"query_string\": \"id=105%27%20UNION%20SELECT%201,username,password%20FROM%20users--\",\n \"attack_type\": \"SQL Injection\",\n \"rule_msg\": \"SQL Injection Attack Detected via libinjection\",\n \"matched_data\": \"UNION SELECT\",\n \"action\": \"blocked\",\n \"http_status\": 403,\n \"request_id\": \"modsec-20250922-144510-8b1a7c\",\n \"user_agent\": \"sqlmap/1.5.2#stable (https://sqlmap.org)\",\n \"tags\": [\n \"application-multi\",\n \"language-multi\",\n \"platform-multi\",\n \"attack-sqli\",\n \"OWASP_CRS/WEB_ATTACK/SQL_INJECTION\"\n]\n }\n}"

Analysis

Description:

At 14:45:10Z, a critical alert was raised by ModSecurity on the host WEB-PAYMENTS-01. The web server detected an SQL Injection attack via the Libinjection check (rule ID 942100). An attacker attempted to exploit a vulnerability in the application running at /products/details.php using a malicious GET request with a tampered query string: 'id=105%27%20UNION%20SELECT%201,username,password%20FROM%20users--'.

Assessment:

This alert indicates an active SQL Injection attack attempt on the payment application hosted on WEB-PAYMENTS-01. The attacker used a UNION SELECT statement to extract sensitive data (usernames and passwords) from the 'users' table in the database. The use of the sqlmap tool (User Agent: sqlmap/1.5.2#stable) suggests that this attack was automated and potentially part of a larger, targeted campaign.

Recommendation:

1. Immediately review and patch any known vulnerabilities on WEB-PAYMENTS-01 to prevent further exploitation.

2. Implement web application firewall (WAF) rules specifically targeting SQL injection attacks to provide additional layers of protection.

3. Analyze the server logs for any other suspicious activities related to this attack, such as failed login attempts or unusual database queries.

4. Reset passwords for all user accounts mentioned in the extracted data (usernames and passwords).

5. Engage with your application development team to review and harden input validation mechanisms against SQL injection attacks.

6. Consider conducting a thorough security audit of the payment application to identify any other potential vulnerabilities."

Judgement:

"description_quality": {
"score": 50,
"error_tags": [
"MISSING_KEY_ENTITY",
"INCORRECT_STATUS"
]
},
"assessment_quality": {
"score": 60,
"error_tags": [
"MISJUDGED_CRITICALITY"
]
},
"recommendation_quality": {
"score": 10,
"error_tags": [
"MISSING_CONTAINMENT",
"HARMFUL_ADVICE",
"GENERIC_ADVICE"

]

},

"thinking_text": "**My Analysis of the LLM's SIEM Alert Evaluation**\n\nOkay, here's the breakdown of my thinking as I evaluate this LLM's response. First, I have to mentally walk through the alert itself, the ground truth, and the model's output. I'm looking at three key areas: description, assessment, and recommendation. I've got to be as precise as possible, so that means scoring it and using error tags.\n\n**Description:** Right off the bat, I see a major red flag. The LLM's description completely omits the crucial detail that the attack was *blocked*. This is a massive issue. It's the difference between a prevented incident and a potential data breach. Sure, it gets the timestamp, host, attack type, and malicious query, but missing that \"blocked\" status is unacceptable. I'm marking it down hard for that, and it gets a low score, around 50. I'm going with `MISSING_KEY_ENTITY` for the missing IP and user-agent. But, definitely `INCORRECT_STATUS` because it misrepresents the actual outcome.\n\n**Assessment:** The LLM does get some things right; it identifies the SQL injection and the intent to grab usernames and passwords, even acknowledging the use of sqlmap. However, it's completely missing the critical context again. Because it missed the \"blocked\" part, its assessment is also off. It's treating this as a possibly successful attempt, which skews its perception of the immediate risk. Also, the assessment is a bit vague, that bit about a \"larger, targeted campaign\" is unsubstantiated. I'm going with around 60 here, and that's generous. I'm using `MISJUDGED_CRITICALITY` for the major flaw.\n\n**Recommendation:** This is where things really fall apart. The recommendations are pretty bad. It's completely missing the immediate containment step of blocking the source IP, which is a total failure! Worse, it recommends resetting passwords. The alert *clearly* says the attack was blocked. This means no data was stolen. Recommending a password reset will cause an operational disruption for no reason and could lead to a loss of user trust. I am giving it a 10 here. I am tagging it with `MISSING_CONTAINMENT`, and absolutely `HARMFUL_ADVICE` because resetting passwords would be disruptive, and wrong. And, many of the other recommendations are just generic, like \"review and patch vulnerabilities\" or \"analyze the server logs.\" I'm tagging those with `GENERIC_ADVICE` because they are not specifically helpful.\n\nAlright, after reviewing the scores, here's my final JSON output and tag assignments:\n```json\n{\n \"description_quality\": {\"score\": 50, \"error_tags\": [\"MISSING_KEY_ENTITY\", \"INCORRECT_STATUS\"]},\n \"assessment_quality\": {\"score\": 60, \"error_tags\": [\"MISJUDGED_CRITICALITY\"]},\n \"recommendation_quality\": {\"score\": 10, \"error_tags\": [\"MISSING_CONTAINMENT\", \"HARMFUL_ADVICE\", \"GENERIC_ADVICE\"]}\n}\n```\nI've checked the tags again. They accurately capture the errors: missing entities, the incorrect statement of status (a major error), misjudged criticality, and absolutely the \"harmful\" nature of the password reset recommendation. Overall, the model needs significant improvement, especially in its understanding of the context and the meaning of blocking an attack.\n"